

ConectaPueblos — Visión Integral de Producto y Evolución del Frontend

Red social territorial para conectar personas, pueblos, experiencias y turismo local

Versión: 1.0 **Fecha:** 2026-07-26 **Alcance:** documento de visión y especificación. No implica implementación. Ningún código, modelo, migración ni endpoint descrito aquí existe todavía salvo que se indique explícitamente como "ya existe" con su ruta de archivo real.

Índice

Resumen ejecutivo

Identidad y principios de producto

Principio de honestidad funcional

Estado actual frente a visión objetivo

Diagnóstico general (fortalezas / debilidades)

Arquitectura de experiencia y navegación

Perfil propio frente a perfil público

Jerarquía detallada por pantalla

Wireframes textuales o estructuras por bloques

Prototipos frontend permitidos

Funciones que no deben activarse sin backend

Confianza, privacidad y moderación (preparación visual)

Accesibilidad y responsive

Sistema de diseño (tokens, componentes, reglas de uso)

Estrategia de fotografía y contenido

Rendimiento percibido

Nuevas funcionalidades propuestas (evaluadas)

Funcionalidades descartadas o pospuestas

Matriz de prioridades

Roadmap integral de frontend (FRONT-0 a FRONT-11)

Dependencias de backend explícitas

Evolución progresiva sin reconstruir el diseño existente

Criterios de aceptación por fase

Decisiones del propietario (resueltas en v1.0)

Arquitectura backend actual relevante

Capacidades backend existentes reutilizables

Brechas backend por funcionalidad

Especificación de endpoints — perfil, onboarding y relaciones entre personas

Especificación de endpoints — Momentos

Especificación de endpoints — Chat y mensajería

Especificación de endpoints — notificaciones, comentarios y reacciones

Especificación de endpoints — publicaciones, tipos de contenido y pueblo enriquecido

Especificación de endpoints — actividades avanzadas, búsqueda, guardados y pasaporte

Modelos, tablas, relaciones e índices necesarios

Migraciones futuras requeridas

Autenticación, autorización y privacidad

Multimedia y almacenamiento

Tiempo real, eventos, notificaciones y jobs

Moderación, seguridad y RGPD

Paginación, consistencia, concurrencia e idempotencia

Estrategia de pruebas backend

Roadmap backend (BACK-0 a BACK-11), matriz de dependencias FRONT↔BACK, orden de implementación full-stack, riesgos y conclusión final

Nota de trazabilidad: el índice aprobado por el propietario numeraba estas mismas secciones del 1 al 42 en un esquema de dos dígitos (26–42 para el capítulo backend). Aquí se mantiene el mismo contenido y orden; la numeración corrida evita duplicar el número "9" en subsecciones y se corresponde 1:1 con lo aprobado.

1. Resumen ejecutivo

ConectaPueblos es una red social territorial: conecta personas a través de pueblos y pueblos a través de personas. La auditoría técnica y de producto realizada sobre el código real (frontend `conecta-pueblos`, backend `Api-ConextaPueblos`) confirma una base sólida — autenticación, catálogos de pueblos y actividades, feed de comunidad, like/save/join/follow — construida con una disciplina poco habitual: **el producto nunca finge tener datos que no tiene**. Ese principio, materializado en el componente `BackendPendingAlert` (`components/ui/backend-pending-alert.tsx`), es un activo de marca y debe seguir siendo el eje de toda evolución futura.

Al mismo tiempo, el producto no es todavía una red social completa entre personas: no existe perfil público, el buscador de cabecera no busca globalmente, no hay comentarios ni notificaciones reales, y funciones ilusionantes (Momentos, Chat, Pasaporte) no existen ni como prototipo. Este documento resuelve esa tensión con un principio ampliado: **la honestidad funcional no prohíbe innovar — prohíbe fingir**. Se puede diseñar y prototipar visualmente el futuro del producto siempre que quede etiquetado sin ambigüedad como demostración, concepto o pendiente de integración.

El documento define: (1) la visión de producto y su jerarquía de pantallas hasta el detalle de bloque; (2) qué puede construirse ya en frontend, qué debe quedar como prototipo visual y qué depende de backend; (3) un roadmap de frontend en 12 fases (FRONT-0 a FRONT-11); y (4) una especificación completa del backend futuro necesario para que cada propuesta se convierta en funcionalidad real, citando siempre el código actual de `Api-ConextaPueblos` como base de partida y marcando explícitamente cualquier endpoint o modelo nuevo como **"Contrato recomendado, todavía no implementado"**.

2. Identidad y principios de producto

Esencia: "Conectar personas a través de los pueblos y conectar pueblos a través de las personas."

Principios rectores:

1. El pueblo es la unidad organizadora del contenido, no el usuario individual. Un post, un Momento, una actividad o una conversación siempre pueden anclarse a un pueblo real.
2. La plataforma no es una guía turística, ni una agencia de viajes, ni una copia de Instagram/Facebook/TikTok. Combina identidad personal, comunidad, vida local, descubrimiento y turismo responsable.
3. La confianza se construye mostrando lo que es real y señalando explícitamente lo que todavía no lo es — nunca simulando.
4. La innovación visual es bienvenida como herramienta de diseño estratégico, siempre que no se presente como funcional sin serlo.
5. Cada pueblo pequeño merece una presencia digital viva, hecha por su comunidad, no solo información institucional.

3. Principio de honestidad funcional

Regla: ninguna pantalla debe mostrar como conectada una función que no tenga backend real y probado.

Aclaración ampliada (v1.0): esto no es una prohibición de innovar en frontend. Se permite y se recomienda diseñar y prototipar experiencias futuras de forma controlada, siempre que:

- Queden etiquetadas explícitamente como demostración, concepto o pendiente de integración, con un aviso visible y permanente (no un texto que desaparece ni una etiqueta secundaria de bajo contraste).
- No utilicen personas, mensajes, contadores, reacciones o actividad que puedan confundirse con datos reales.
- No sean accesibles desde la navegación principal como si fueran una función operativa.
- Convivan con dos patrones de UI distintos y reconocibles en el sistema de diseño:
- **BackendPendingAlert** (ya existe, `components/ui/backend-pending-alert.tsx`): "esto no existe todavía", usado en rutas reales sin función.
- **Banner de prototipo** (nuevo, a introducir): "esto es una vista previa de diseño, no funcional, sin datos ni personas reales", usado en las vistas conceptuales de Momentos y Chat.

4. Estado actual frente a visión objetivo

Dimensión	Estado actual	Visión objetivo
Unidad social	Pueblo con muro y actividades; usuario sin red social entre personas	Pueblo + red de personas (perfil público, seguimiento entre usuarios)
Descubrimiento	Catálogos separados de pueblos/actividades con filtros locales; <code>/explore</code> vacío; buscador de cabecera solo filtra el feed	Búsqueda unificada real, landing con contenido real, <code>/explore</code> funcional
Contenido	Un tipo de publicación (texto + imagen opcional), sin comentarios	Publicaciones tipadas, comentarios, Momentos como capa efímera diferenciada
Identidad personal	Perfil propio con 3 métricas y 3 tabs	Perfil propio ampliado (portada, intereses, pasaporte, fijados) + perfil público con reglas de visibilidad
Comunicación	Sin mensajería ni notificaciones reales	Chat 1:1 real, notificaciones reales con badge
Memoria territorial	No existe	Pasaporte de pueblos (seguidos, quiero visitar, visitados)
Confianza	Sin reportes ni bloqueo	Reportar/bloquear en todas las superficies sociales, con moderación real

5. Diagnóstico general

Fortalezas actuales

- Honestidad de producto (`BackendPendingAlert`) reutilizada en Mensajes, Notificaciones, Guardados, Explorar, Admin y detalle de actividad.
- Accesibilidad real: focus trap completo (`components/ui/use-modal-dialog.ts`), roving tabindex en tabs (`features/notifications/notifications-view.tsx`), `aria-live` correcto (`components/ui/app-toast.tsx`), `alt` diferenciado real/decorativo.
- Identidad territorial en el copy de navegación (`components/layout/navigation-items.ts`).
- Hero de pueblo logrado y a conservar (`app/villages/[id]/page.tsx`).
- Patrón de interacción optimista unificado y reutilizable (`components/social/use-keyed-optimistic-boolean.ts`).
- 100% `next/image`, respeto a `prefers-reduced-motion` (`app/globals.css`).

Debilidades actuales

- No existe perfil público (ningún autor es un enlace; sin ruta `/u/[username]`).
- El buscador de cabecera (`components/layout/navbar.tsx`) es percibido como global pero solo filtra `/community`; `/explore` está vacío.
- Cero onboarding: de registro a `/community` sin ningún paso intermedio.
- Ajustes es un único formulario plano, sin secciones de privacidad/seguridad/notificaciones.
- Sin indicador de actividad pasada o aforo lleno — bug de confianza reproducible hoy mismo.
- Navegación inconsistente entre breakpoints (sidebar desktop oculta Mensajes/Notificaciones/Ajustes que sí están en el drawer móvil).
- Tokens de color literales (`--forest`, `--terracotta`) sin capa semántica — bloquean tema oscuro sin `*find-replace*` global.
- Sin paginación en ningún listado (`limit=100` fijo en `lib/api/*.service.ts`) — corte silencioso al superar 100 elementos.

6. Arquitectura de experiencia y navegación

Desktop: Navbar (logo, buscador de feed visible solo en rutas sociales desde `md:`, icono de Mensajes `lg:grid`, campana de notificaciones, menú de usuario) + Sidebar izquierdo fijo (Comunidad/Actividades/Pueblos, y si hay sesión: Perfil/Explorar/Guardados) + Rail derecho opcional solo en `/community` (Actividades para descubrir, Pueblos para descubrir,

"Fotos recientes de la comunidad").

Móvil: Navbar sin buscador visible + Bottom nav fijo de 4 accesos (Comunidad/Planes/Pueblos/Perfil) + Drawer hamburguesa con los accesos secundarios completos (Perfil, Explorar, Guardados, Mensajes, Notificaciones, Ajustes).

Corrección prioritaria (FRONT-0): unificar qué accesos secundarios existen en cada breakpoint (hoy el sidebar desktop solo replica 3 de los 6) y hacer visible un acceso de búsqueda también en móvil.

7. Perfil propio frente a perfil público

Perfil propio — elementos y su naturaleza

Elemento	Visibilidad	Estado
Portada / avatar	Pública, configurable	Funcional hoy (<code>features/profile/settings-view.tsx</code>)
Biografía	Pública, configurable	Funcional hoy
Pueblo de origen y de residencia	Configurable (público solo si el usuario lo decide)	Pendiente de backend — campo nuevo, distinto de <code>favorite_village_id</code>
Intereses	Configurable	Pendiente de backend
Publicaciones	Pública (según reglas del post)	Funcional hoy
Fotografías	Pública	Prototipo de UI posible hoy reutilizando <code>image_url</code> de posts propios, sin backend nuevo
Recuerdos	Pública	Pendiente de backend (tipo/metadata distinta a un post)
Actividades / Pueblos (organizadas/seguidos)	Configurable	Funcional con limitación (sin endpoint dedicado)
Pasaporte	Configurable	Prototipo funcional real (reutiliza follow/save)
Pueblos pendientes ("Quiero visitar")	Privada por defecto, configurable	No se infiere de guardado/seguido — relación propia nueva, pendiente de backend (ver §9.5 y Decisión 3)
Contenido fijado	Pública	Pendiente de backend (campo <code>pinned</code>); prototipable visualmente, no persistente sin él
Momentos destacados	Pública	Doble dependencia: backend de Momentos + backend de perfil — fase futura
Guardados privados	Siempre privada	Acción funcional hoy; listado pendiente de backend
Controles de visibilidad por sección	—	Se diseña ahora; no se activa hasta que el backend restrinja de verdad (evita un selector de privacidad falso)

Perfil público — reglas de v1.0 (decisión del propietario)

Visible: nombre, username, avatar, portada, biografía, intereses, publicaciones públicas, fotografías procedentes de publicaciones públicas, actividades organizadas, pueblo de origen o residencia **solo si el usuario decide hacerlo visible**.

Privado siempre: correo, rol interno, guardados, borradores, historial de navegación, actividades inscritas (por defecto), ubicación precisa.

Diseñables pero no activables sin backend: botón "Seguir" y botón "Mensaje" — se muestran en el diseño, deshabilitados o con acción interceptada (`BackendPendingAlert`), hasta que exista seguimiento entre usuarios (§9.5) y chat (§9.4 / §30).

8. Jerarquía detallada por pantalla

8.1 Landing

Hero de marca (`features/auth/public-auth-shell.tsx`) sólido pero 100% estático. Mejora funcional-hoy: sustituir las 3 tarjetas de marketing hardcodeadas por datos reales ya servidos públicamente (`getVillagesStrict` / `getActivitiesStrict`), y añadir CTA secundaria "Explorar sin cuenta" hacia `/villages` .

8.2 Onboarding

No existe. Propuesta mínima de 2 pasos, saltables, sin campos backend nuevos en v1: paso 1 "elige 1-3 pueblos que te interesan" (usa `followVillage` real); paso 2 "completa tu perfil" (usa `updateCurrentUser` real). Intereses y pueblo de origen/residencia quedan preparados en el formulario pero marcados pendientes hasta que existan esos campos en backend (§28).

8.3 Comunidad y feed

Un tipo de publicación real (texto + imagen opcional + pueblo). El filtro "Avisos/Actividades" actual es una heurística de texto, no un campo real — no debe ampliarse sin antes tener `type` real en backend (§32).

8.4 Perfil

Ver §7. Estructura de pantalla: cabecera (portada, avatar, nombre, pueblo si es visible, bio, métricas) → tabs (Publicaciones, Fotografías, Actividades, Pueblos, Pasaporte) → rail de recomendaciones.

8.5 Página de pueblo (hero intacto — no se toca)

Jerarquía completa de la parte inferior, en el orden aprobado:

#	Bloque	Presentación	Fuente de datos	Naturaleza
1	Estadísticas	Directo (fact-cards)	Real, ya existe	Funcional
2	Sobre el pueblo	Directo, expandible si el texto es largo	<code>description</code> , real	Funcional
3	Qué hace especial al pueblo	Directo si hay <code>highlights</code> ; oculto si no hay dato	<code>highlights</code> , real	Funcional condicional
4	Vida actual	Resumen directo ("N actividades próximas, N publicaciones esta semana"), enlaza a bloques 5 y 7	Derivado en frontend de datos ya cargados	Funcional (agregador)
5	Actividades	Grid directo; carrusel si hay más de ~6	Real, ya existe	Funcional
6	Muro	Lista vertical directa (cronológica)	Real, ya existe	Funcional, limitado por falta de paginación
7	Voces de la comunidad	Directo, curaduría de 2-4 publicaciones reales del muro	Real (subconjunto del muro)	Funcional, con advertencia: no afirma que el autor es habitante mientras no exista dato verificable de residencia/origen (§7)
8	Galería / "Fotos recientes de la comunidad"	Carrusel directo, nombre preciso (no "recuerdos" ni "galería histórica", que implicarían curaduría inexistente)	Reutiliza <code>image_url</code> real de posts del pueblo	Funcional, re-etiquetado
9	Información práctica	Directo con dato real disponible (pueblo/provincia/región/población); campos sin dato real no se muestran	Real parcial	Funcional parcial; ampliación (transporte, mejor época, enlaces oficiales) pendiente de backend
10	Pueblos relacionados	Carrusel directo, calculado en frontend filtrando el catálogo ya cargado por región/provincia	Derivado, sin backend nuevo	Funcional
11	Acciones para participar	Directo: seguir, ver actividades, "Publicar aquí"/"Recomendar"/"Preguntar" (abren el composer real preconfigurado, sin tipos de post nuevos)	Reutiliza composer real	Funcional

8.6 Actividades

Falta reproducible: sin badge de "Finalizada"/"Completo" pese a que `spotsLeft` /fecha ya están disponibles; sin filtro de fecha en el explorador pese a que el backend ya lo soporta (`date_from` / `date_to`). Ambas correcciones son 100% frontend.

8.7 Momentos (conceptual)

Ver §10 y §29. No aparece en navegación principal. Nombre de concepto general: **Momentos**; formatos internos: Ahora, Postal, Recuerdo, Plan, Consejo, Aviso.

8.8 Chat (prototipo visual)

Ver §10 y §30. Estructura: lista de conversaciones (desktop) / navegación de pantalla completa (móvil), conversación activa, panel contextual (persona, pueblo o actividad compartida).

8.9 Notificaciones

Tabs accesibles ya construidas (`features/notifications/notifications-view.tsx`); falta badge de no leídas en la campana (`components/ui/notification-bell.tsx`) — pendiente de backend para tener un número real que mostrar.

8.10 Configuración

Migrar de formulario único a secciones (Cuenta, Perfil, Privacidad, Notificaciones, Seguridad, Accesibilidad), con controles de visibilidad del perfil diseñados pero marcados pendientes.

8.11 Búsqueda

Unificar el componente `SearchInput` (hoy solo en `features/villages/village-explorer.tsx`) en pueblos, actividades y comunidad; renombrar con precisión el input de la navbar ("Buscar en publicaciones") mientras `/explore` no sea una búsqueda real.

8.12 Guardados y colecciones

Los mutadores ya funcionan; falta el listado. Colecciones temáticas ("Próximos viajes", "Escapadas") se posponen hasta que exista el listado base.

9. Wireframes textuales o estructuras por bloques

Perfil propio (desktop, de arriba a abajo):

[Portada 240px] [Avatar 96px superpuesto]
 [Nombre] [Pueblo (si visible)] [Editar perfil]
 [Bio]
 [Publicaciones N] [Actividades N] [Pueblos N] [Pasaporte N]
 [Tabs: Publicaciones | Fotografías | Actividades | Pueblos | Pasaporte]
 [Contenido de la tab activa – grid o lista]

Página de pueblo (parte inferior, desktop en 2 columnas + franjas de ancho completo):

[Hero – sin cambios]
 [Estadísticas – 4 fact-cards en fila]
 [Columna izq: Sobre el pueblo → Qué lo hace especial → Vida actual] [Columna der: Ficha local → Señas del pueblo]
 [Muro – ancho completo, lista vertical]
 [Voces de la comunidad – ancho completo, 2-4 tarjetas destacadas]
 [Fotos recientes de la comunidad – carrusel ancho completo]
 [Actividades en el pueblo – grid ancho completo]
 [Pueblos relacionados – carrusel]
 [Acciones para participar – barra fija o tarjeta final]

Momentos (prototipo, pantalla de visor):

```
[Banner permanente: "Vista previa conceptual – sin datos reales"]
[Anillo de autor – ejemplo etiquetado]
[Visor de contenido – imagen/texto de ejemplo con marca de agua "Ejemplo"]
[Tipo: Ahora | Postal | Recuerdo | Plan | Consejo | Aviso – chip visible]
[Controles deshabilitados: reacción, respuesta – con tooltip "Pendiente de backend"]
```

Chat (prototipo, desktop 3 columnas):

```
[Banner permanente: "Vista previa conceptual – no operativo"]
[Columna 1: lista de conversaciones – filas de ejemplo, sin nombres reales]
[Columna 2: conversación activa – burbujas de ejemplo, input deshabilitado]
[Columna 3: panel contextual – persona / pueblo / actividad compartida]
```

Móvil: navegación de pantalla completa entre columna 1 y 2; columna 3 como hoja inferior.

10. Prototipos frontend permitidos

Se autorizan explícitamente como prototipos visuales, bajo el banner permanente descrito en §3:

- **Momentos:** visor conceptual, no accesible desde navegación principal, sin reacciones/respuestas reales.
- **Chat:** vista de conversaciones/panel contextual/responsive, accesible únicamente desde `/messages` mediante una acción secundaria explícita **"Ver propuesta visual"**, manteniendo la ruta real en su estado honesto de `BackendPendingAlert` por defecto.
- **Fotografías del perfil propio:** reutilizar imágenes reales de posts propios en una cuadrícula, sin backend nuevo.

No se autoriza ningún prototipo que use nombres, avatares o mensajes que puedan confundirse con personas o contenido real.

11. Funciones que no deben activarse sin backend

Comentarios, compartir, notificaciones reales, mensajería operativa, Momentos operativos, seguimiento entre usuarios, colecciones guardadas/pasaporte con persistencia real, perfil público con datos reales de terceros, pueblo favorito persistente, subida real de archivos, recuperación de contraseña, búsqueda global real, panel de moderación, badge de no leídas con número real.

12. Confianza, privacidad y moderación (preparación visual)

Añadir un menú "..." en `social-post-card.tsx` con "Reportar" interceptado (mismo patrón que Foto/Aviso en el composer) — barato ahora, mucho más caro de insertar después sin rehacer el layout de la card. Mismo patrón aplicable al prototipo de Chat y al futuro perfil público (botón "Bloquear" visible, interceptado).

13. Accesibilidad y responsive

Mantener el estándar ya alcanzado (focus trap, roving tabindex, `aria-live`, `alt` diferenciado) en toda pantalla nueva. Pendiente de ejecutar (no code): matriz manual 375–1440px + teclado + lector de pantalla sobre las pantallas nuevas antes de cada release de fase.

14. Sistema de diseño (tokens, componentes, reglas de uso)

`app/globals.css` usa Tailwind v4 config-less con tokens **literales** (`--forest`, `--terracotta`, `--mustard` ...), sin modo oscuro. Recomendación FRONT-0: introducir una capa semántica de alias (`--color-primary: var(--forest)`, `--color-accent: var(--terracotta)`, etc.) sin cambiar los valores actuales — permite theming futuro sin `*find-replace*` global y sin

alterar la identidad visual existente. Documentar la regla de uso de las 5 variantes de botón (`primary/secondary/ghost/terracotta/mustard`) antes de añadir una sexta. Mantener `category-pill.tsx` como está — candidato a limpieza futura, no se toca en este ciclo.

15. Estrategia de fotografía y contenido

Distinguir siempre tres tipos de imagen en la interfaz: **contenido real de usuario** (posts, perfil), **imagen editorial de respaldo** (ya implementado con badge "Imagen editorial" en `features/activities/activity-image.tsx`) y **contenido de ejemplo en prototipos** (marca de agua o etiqueta "Ejemplo", nunca fotografías que parezcan reales de personas). Tono de contenido: humano, cercano, comunitario — evitar superlativos turísticos genéricos.

16. Rendimiento percibido

100% `next/image` ya logrado. Sin paginación en ningún listado (`limit=100` fijo) — mientras no exista paginación backend, mostrar un aviso "mostrando los primeros 100 resultados" en vez de un corte silencioso.

17. Nuevas funcionalidades propuestas (evaluadas)

Propuesta	Valor	Dependencia	Prioridad
Perfil público (v1 con reglas de §7)	Muy alta	Backend nuevo (§28)	Muy alta
Unificar buscador	Alta	Ninguna	Alta
Badges pasada/llena en actividades	Alta	Ninguna	Alta
Filtro de fecha en actividades	Media-alta	Ninguna (backend ya lo soporta)	Alta
Onboarding 2 pasos	Alta	Ninguna	Alta
Agenda personal (.ics)	Media	Ninguna	Media
Pasaporte v1	Alta	Ninguna (reutiliza follow/save)	Media
"Pueblos silenciosos"	Media-alta	Parcial — ver limitación (§17.1)	Media
Momentos (prototipo → operativo)	Alta (diferenciador)	Backend nuevo (§29)	Media (prototipo ya)
Chat 1:1	Alta	Backend nuevo (§30)	Media (prototipo ya)
Multilingüe	Media	Librería i18n	Baja (FRONT-11)
Perfiles de asociaciones/comercios	Alta (impacto económico)	Backend (rol/entidad nueva)	Baja (fase futura)

17.1 Limitación de "pueblos silenciosos"

Los listados actuales no exponen `total` ni paginación completa (tope de 100 elementos) y los timestamps backend no garantizan zona horaria fiable. Por tanto, "pueblo silencioso" solo puede calcularse hoy como una **aproximación sobre la página cargada** ("sin publicaciones recientes en los últimos resultados cargados"), nunca como una afirmación certera de inactividad total, hasta que exista paginación y fechas fiables en backend (§40).

18. Funcionalidades descartadas o pospuestas

- 10 tipos de publicación simultáneos → colapsar a Publicación + Aviso (con `type` real).
- Chat en grupo → pospuesto hasta validar 1:1 operativo.
- Pasaporte con sellos verificados por geolocalización → v2, fase futura.

- "Pueblos pendientes" inferido de guardado/seguído → descartado explícitamente; se documenta como relación propia nueva "Quiero visitar" (§28).

19. Matriz de prioridades

Muy alta / ahora, 100% frontend: perfil propio ampliado, activación de perfil público (bloqueada por backend §28), unificar búsqueda, badges pasada/llena, landing con datos reales, onboarding mínimo. **Alta:** reestructurar Ajustes, filtro de fecha en actividades, menú "Reportar" interceptado, unificación de accesos entre breakpoints. **Media:** agenda .ics, Pasaporte v1, prototipos de Momentos y Chat, tokens semánticos. **Baja / futuro:** multilingüe, perfiles de asociaciones/comercios, mapa geo, Momentos y Chat operativos (dependen de backend).

20. Roadmap integral de frontend (FRONT-0 a FRONT-11)

Cada fase indica su naturaleza: **Funcional** (datos y backend reales) · **Prototipo visual** (diseño/demo etiquetado) · **Bloqueado por backend** · **Fase futura**.

Fase	Objetivo	Pantallas / componentes	Naturaleza	Prioridad
FRONT-0	Navegación consistente, búsqueda unificada, capa semántica de tokens	navbar.tsx, sidebar-nav.tsx, mobile-bottom-nav.tsx, responsive-sidebar-drawer.tsx, SearchInput	Funcional	Muy alta
FRONT-1	Perfil propio ampliado + activación de perfil público	profile-view.tsx, profile-header.tsx, profile-tabs.tsx, nueva ruta /u/[username]	Funcional, bloques pendientes marcados	Muy alta
FRONT-2	Página de pueblo — jerarquía de 11 bloques	app/villages/[id]/page.tsx	Funcional en su mayoría	Alta
FRONT-3	Comunidad: badges pasada/llena, filtro fecha, menú reportar	activity-card.tsx, activity-explorer.tsx, social-post-card.tsx	Funcional	Alta
FRONT-4	Descubrimiento: landing con datos reales, /explore unificado	app/page.tsx, public-auth-shell.tsx, app/explore/page.tsx	Funcional	Alta
FRONT-5	Configuración reestructurada en secciones	features/profile/settings-view.tsx	Funcional + controles de privacidad bloqueados	Alta
FRONT-6	Pasaporte v1 ("Mi mapa de pueblos")	Nueva tab en perfil	Funcional (con limitación §17.1)	Media
FRONT-7	Chat — prototipo visual completo	Nueva vista enlazada desde /messages	Prototipo visual, bloqueado para operar	Media
FRONT-8	Momentos — prototipo aislado	Nueva vista, sin nav principal	Prototipo visual, bloqueado para operar	Media
FRONT-9	Confianza y moderación visual (reportar/bloquear)	social-post-card.tsx, perfil público, chat prototipo	Interceptores funcionales; moderación real bloqueada	Media
FRONT-10	Pulido global: responsive, accesibilidad, rendimiento	Transversal	Funcional	Continuo
FRONT-11	Multilingüe (catalán/castellano)	Transversal	Fase futura	Baja

21. Dependencias de backend explícitas

Perfil público → §28 · Onboarding/preferencias → §28 · Pasaporte real ("Quiero visitar") → §28 · Momentos operativos → §29 · Chat operativo → §30 · Notificaciones/comentarios → §31 · Publicaciones tipadas y pueblo enriquecido → §32 · Actividades avanzadas/búsqueda/guardados → §33.

22. Evolución progresiva sin reconstruir el diseño existente

Todas las fases reutilizan componentes ya existentes (`Card`, `Button`, `BackendPendingAlert`, `use-keyed-optimistic-boolean`, `use-modal-dialog`) y el hero de pueblo permanece intacto. Ninguna fase requiere rediseñar el sistema visual base — solo extenderlo con una capa semántica de tokens (§14) y nuevos patrones de "prototipo etiquetado" (§3).

23. Criterios de aceptación por fase

- **FRONT-0:** los mismos 6 accesos secundarios aparecen en desktop y móvil; búsqueda visible en ambos; ningún token de color nuevo sin alias semántico.
- **FRONT-1:** perfil público accesible desde cualquier autor de post; campos privados nunca presentes en el HTML renderizado (no solo ocultos por CSS).
- **FRONT-2:** ninguna sección del pueblo muestra dato inventado; "Voces de la comunidad" nunca afirma residencia no verificada.
- **FRONT-6:** Pasaporte muestra el texto de limitación de §17.1 cuando corresponda.
- **FRONT-7 / FRONT-8:** el banner de prototipo es ineliminable por el usuario y persiste en toda la sesión de esa vista.

24. Decisiones del propietario (resueltas en v1.0)

1. Nombre de la función efímera: **Momentos** (concepto general); formatos: Ahora, Postal, Recuerdo, Plan, Consejo, Aviso.
2. Perfil público v1: campos visibles/privados/configurables según §7.
3. "Pueblos pendientes de visitar" = relación propia nueva "Quiero visitar", nunca inferida de guardado/seguído.
4. Chat y Momentos = vista previa conceptual; rutas reales conservan `BackendPendingAlert` con acción secundaria "Ver propuesta visual".
5. `category-pill.tsx` se conserva; candidato a limpieza futura.
6. Multilingüe confirmado como FRONT-11, fase futura.
7. Sección del pueblo: "Voces de la comunidad", sin afirmar residencia no verificada.

Especificación backend futura

Todo lo descrito desde aquí que no cite una ruta de archivo existente es un **contrato recomendado, todavía no implementado**. No se ha modificado, creado ni ejecutado ningún archivo del backend para producir este documento.

25. Arquitectura backend actual relevante

Stack confirmado en `requirements.txt`: `FastAPI`, `uvicorn[standard]`, `pydantic-settings`, `python-dotenv`, `SQLAlchemy`, `psycopg2-binary`, `Alembic`, `python-jose[cryptography]`, `passlib[bcrypt]` + `bcrypt==4.0.1`, `python-multipart`, `email-validator`, `eval_type_backport`. **No hay ninguna librería de almacenamiento externo (boto3/S3), tiempo real (websockets, python-socketio) ni cola/caché (redis, celery)** — cualquier función de esa naturaleza es 100% nueva.

Capas: `app/api/v1/endpoints/*.py` (routers) → `app/services/*.py` (lógica) → `app/models/*.py` (SQLAlchemy 2.0, `Mapped / mapped_column`) → `app/schemas/*.py` (Pydantic v2). Sin capa de repositorio; los servicios consultan el ORM directamente. Autenticación: esquema Bearer manual (no `OAuth2PasswordBearer`) en `app/api/v1/endpoints/auth.py`, JWT vía `python-jose`, hashing vía `passlib/bcrypt` (`app/core/security.py`).

Modelos existentes (`app/models/`): `user.py` , `village.py` , `activity.py` , `community_post.py` , `interactions.py` (contiene `VillageFollower` , `ActivityParticipant` , `SavedActivity` , `PostLike` , `SavedPost`). Todos con `id` UUID, soft delete vía `deleted_at` , y las tablas de interacción con `id` , `<padre>_id` FK CASCADE, `user_id` FK CASCADE, `created_at` y restricción única por par.

Endpoints existentes (`app/api/v1/endpoints/`): `health.py` , `auth.py` , `users.py` , `villages.py` , `activities.py` , `community_posts.py` — cubren registro/login/perfil, CRUD de pueblos (admin), CRUD de actividades (organizador/admin) con join/save, CRUD de posts (autor/admin) con like/save.

Migraciones existentes (`alembic/versions/` , 6 revisiones): `8cb495cde376_create_users_table.py` → `71364bd57711_create_villages_table.py` → `552254c58d8b_create_community_posts_table.py` → `49a38d4c7123_create_activities_table.py` → `d081bc3d554e_create_interaction_tables.py` → `f1c3a8d9b120_add_community_posts_created_at_index.py` .

Deuda técnica ya señalada por la auditoría técnica de este mismo ciclo (no repetida en detalle aquí): `favorite_village_id` se acepta y descarta sin persistir (`app/api/v1/endpoints/users.py`); GET de villages/activities/posts abren `SessionLocal()` manual en vez de `Depends(get_db)` ; `response_model` no está aplicado a las respuestas de interacción (`app/schemas/common.py` define `FollowResponse` / `JoinResponse` / `SaveResponse` / `LikeResponse` sin uso real); `comments_count` / `shares_count` existen como columnas muertas en `community_post.py` .

26. Capacidades backend existentes reutilizables

Sin cambios: autenticación JWT, hashing de contraseña, CRUD de pueblos/actividades/posts, follow de pueblo, join/leave de actividad, save/unsave de posts y actividades, like/unlike de posts, filtros de búsqueda por texto/categoría/pueblo/fecha en activities, paginación por `offset` / `limit` (sin `total`). Estas capacidades son la base sobre la que se construye todo lo nuevo — **no se propone sustituir ninguna**, solo extenderlas.

27. Brechas backend por funcionalidad

Funcionalidad propuesta	Tipo de brecha
Perfil público con reglas de visibilidad	Existe endpoint sin uso (<code>GET /users/{id_or_username}</code> , señalado en auditoría técnica previa) pero sin separación público/privado — debe ampliarse
Pueblo de origen/residencia, intereses	No existe — campo nuevo
Seguimiento entre usuarios	No existe — entidad nueva
Momentos	No existe — dominio nuevo completo
Chat	No existe — dominio nuevo completo
Comentarios	No existe — dominio nuevo completo
Notificaciones	No existe — dominio nuevo completo
Publicaciones tipadas	Existe <code>CommunityPost</code> de un solo tipo — debe ampliarse con <code>type</code>
Multimedia (upload real)	No existe — dominio nuevo completo, requiere almacenamiento externo
"Quiero visitar" / Pasaporte real	No existe — entidad nueva, distinta de <code>SavedActivity</code> / <code>VillageFollower</code>
Colecciones guardadas (listado)	Mutadores existen; falta el GET de colección
Búsqueda global	No existe — requiere índices nuevos
Pueblo enriquecido (historia, gastronomía, rutas...)	Campos parciales en <code>village.py</code> (<code>description</code> , <code>highlights</code> JSONB); resto no existe

28. Especificación de endpoints — perfil, onboarding y relaciones entre personas

Ficha: Perfil público con visibilidad por campo

- **Estado actual:** GET `/api/v1/users/{id_or_username}` existe (`app/api/v1/endpoints/users.py`) pero devuelve el mismo `UserResponse` sin distinguir qué es público; expone `role` sin decisión de privacidad.
- **Capacidad reutilizable:** el endpoint y la resolución por id/username ya existen; solo cambia el modelo de respuesta.
- **Endpoint nuevo recomendado:** ampliar el existente, no crear uno paralelo. *Contrato recomendado, todavía no implementado.*
- **Método/ruta:** GET `/api/v1/users/{id_or_username}` (reutilizado)
- **Auth:** opcional (público)
- **Request:** ninguno (path param).
- **Response (`UserPublicResponse` , nuevo schema):** `{id, name, username, avatar_url, banner_url, bio, interests[], home_village (si visible), organized_activities_count, posts_count, followers_count}`. **Nunca** incluye `email`, `role`, `password_hash`, `saved_*`, `drafts`.
- **Códigos:** 200, 404 (usuario inexistente/soft-deleted).
- **Validaciones:** ninguna de entrada; en salida, filtrar por `user_privacy_settings` (ficha siguiente).
- **Modelo/tabla/migración:** requiere tabla nueva `user_privacy_settings` (o columnas booleanas en `users` : `show_home_village`, `show_interests`) — migración Alembic nueva.
- **Servicio:** nuevo `UserPublicService` que compone `UserPublicResponse` respetando la tabla de privacidad.
- **Privacidad/moderación:** central para esta ficha — es la razón de ser del endpoint.
- **Pruebas:** un usuario con `show_home_village=false` nunca debe devolver ese campo, verificado por test de contrato, no solo por inspección visual.
- **Dependencias:** requiere §28 "Preferencias de privacidad" y §34 (columna `home_village_id`).
- **Prioridad / fase:** Alta / BACK-1.

Ficha: Pueblo de origen, residencia e intereses

- **Estado actual:** no existe ningún campo en `app/models/user.py` más allá de `avatar_url`, `banner_url`, `bio`, `role` .
- **Endpoint nuevo recomendado:** ampliar PUT `/api/v1/users/me` (reutilizado, ya existe y ya persiste `name/username/avatar_url/banner_url/bio`).
- **Request:** añadir `home_village_id (uuid|null)`, `residence_village_id (uuid|null)`, `interests (string[], máx. 10)`, `show_home_village (bool)` .
- **Response:** `UserMeResponse` ampliado con los mismos campos.
- **Códigos:** 200, 401, 404 (village inexistente), 422.
- **Validaciones:** `home_village_id / residence_village_id` deben existir y no estar soft-deleted; `interests` de una lista controlada (tabla `interest_tags` o enum) para evitar texto libre sin moderar.
- **Modelo/tabla/migración:** columnas nuevas en `users` (`home_village_id FK villages ON DELETE SET NULL` , `residence_village_id FK villages ON DELETE SET NULL` , `show_home_village boolean default false`) + tabla `user_interests (user_id, interest_tag_id)` si se opta por catálogo controlado — **migración Alembic nueva**, corrige de paso el bug ya conocido de `favorite_village_id` (se recomienda resolverlo en la misma migración, no crear un tercer campo de pueblo).
- **Dependencias:** ninguna externa.
- **Prioridad / fase:** Alta / BACK-1.

Ficha: Seguimiento entre usuarios

- **Estado actual:** no existe. Solo existe `VillageFollower` (seguimiento a pueblos, no a personas).
- **Endpoints nuevos:** *Contrato recomendado, todavía no implementado.*
- POST `/api/v1/users/{id_or_username}/follow` · Bearer · body vacío · response `{following: true, followers_count}` · 200, 401, 404, 409 (auto-follow)
- DELETE `/api/v1/users/{id_or_username}/follow` · igual, `following: false`

- `GET /api/v1/users/{id_or_username}/followers` · público según privacidad · paginado
- `GET /api/v1/users/{id_or_username}/following` · público según privacidad · paginado
- **Modelo/tabla:** `user_followers` (`id`, `follower_id` FK `users` CASCADE, `followed_id` FK `users` CASCADE, `created_at`, `UNIQUE(follower_id, followed_id)`), índice compuesto.
- **Validaciones:** prohibir `follower_id == followed_id` (409); respetar bloqueos (§39) antes de permitir follow.
- **Eventos:** genera notificación tipo `new_follower` (§38).
- **Pruebas:** idempotencia, aislamiento entre usuarios, 409 en auto-follow, bloqueo impide follow.
- **Prioridad / fase:** Media / BACK-1.

Ficha: Preferencias de onboarding

- **Estado actual:** no existe ningún campo ni endpoint de onboarding.
- **Endpoints nuevos:** *Contrato recomendado, todavía no implementado.*
- `GET /api/v1/users/me/onboarding` · Bearer · response `{completed: bool, steps_skipped: string[]}`
- `PATCH /api/v1/users/me/onboarding` · Bearer · body `{completed: bool, steps_skipped: string[]}` · 200, 401, 422
- `GET /api/v1/users/me/onboarding/suggestions` · Bearer · response `{villages: VillagePublicResponse[]}` (reutiliza `GET /villages` con un `limit` bajo y orden por popularidad — sin lógica nueva de recomendación en v1)
- **Modelo/tabla:** columna `onboarding_completed` boolean default `false` + `onboarding_skipped_steps` `text[]` en `users`, o tabla `user_onboarding_state` si se prevé más de 2-3 pasos.
- **Dependencias:** ninguna — puede construirse en paralelo a todo lo demás.
- **Prioridad / fase:** Media / BACK-1.

29. Especificación de endpoints — Momentos

Estado actual: no existe ningún modelo, tabla ni endpoint. Dominio nuevo completo.

Ficha: Crear Momento

- **Endpoint:** `POST /api/v1/moments` · Bearer · *Contrato recomendado, todavía no implementado.*
- **Request:** `{type: "ahora"|"postal"|"recuerdo"|"plan"|"consejo"|"aviso", content (texto, opcional según tipo), media_id (uuid, opcional, ver §37), village_id (uuid, opcional), activity_id (uuid, opcional), visibility: "public"|"followers"}`
- **Response:** `{id, type, author, village, activity, media_url, content, visibility, expires_at, created_at}`
- **Códigos:** 201, 401, 404 (village/activity/media inexistente), 422.
- **Validaciones:** `content` obligatorio si no hay `media_id`; `expires_at = created_at + 24h` calculado en servidor, nunca enviado por el cliente; tipo `aviso` puede requerir rol/verificación adicional (a decidir en fase de moderación).
- **Modelo/tabla:** `moments` (`id`, `author_id` FK `users` CASCADE, `type` enum, `content`, `media_id` FK `media_assets` nullable, `village_id` FK `villages` nullable, `activity_id` FK `activities` nullable, `visibility` enum, `expires_at` indexed, `created_at`, `deleted_at`). Índice en `(expires_at)` para el job de expiración y en `(author_id, created_at)`.
- **Migración:** nueva, crea tabla `moments` + enum `moment_type`.
- **Servicio:** `MomentService.create`, valida ownership de `village_id` / `activity_id` como referencias, no como control de acceso.
- **Eventos:** ninguno al crear (las reacciones/respuestas si generan notificación, ver ficha siguiente).
- **Almacenamiento:** requiere `media_assets` (§37) si el Momento incluye imagen/vídeo.
- **Prioridad / fase:** Alta (para desbloquear FRONT-8) / BACK-9.

Ficha: Expiración, vistas, reacciones y respuestas

- **Endpoints:** *Contrato recomendado, todavía no implementado.*
- `GET /api/v1/moments/feed` · Bearer opcional · paginado · devuelve Momentos no expirados (`expires_at > now()`) de pueblos/personas seguidas

- `POST /api/v1/moments/{id}/views` · Bearer · idempotente por `(moment_id, user_id)` · no incrementa en visitas repetidas del mismo usuario
- `POST /api/v1/moments/{id}/reactions` · Bearer · body `{emoji}` · 200/201
- `POST /api/v1/moments/{id}/replies` · Bearer · body `{content}` · genera notificación al autor (§38)
- `DELETE /api/v1/moments/{id}` · Bearer autor/admin · soft delete inmediato (no espera a expiración)
- **Tablas:** `moment_views (moment_id, user_id, viewed_at, UNIQUE(moment_id, user_id))`, `moment_reactions (moment_id, user_id, emoji, created_at, UNIQUE(moment_id, user_id))`, `moment_replies (id, moment_id, author_id, content, created_at, deleted_at)`, `moment_highlights (id, user_id, moment_id, created_at)` para "destacados" de perfil.
- **Expiración — qué se borra físicamente y qué solo deja de mostrarse:** a las 24h, el Momento **deja de aparecer en el feed** (filtro `expires_at > now()` en la query, sin borrado físico) para permitir que el autor lo mueva a "Destacados" (`moment_highlights`) antes de que un job lo purgue. Un **job programado** (nuevo, requiere scheduler — no existe hoy ningún mecanismo de jobs en el backend) purga físicamente (borra fila + `media_assets` asociado si no está en ningún `moment_highlights`) pasadas, por ejemplo, 72h desde `expires_at`, dando margen a destacar. Los que sí están en `moment_highlights` se conservan indefinidamente.
- **Tiempo real:** v1 puede resolverse con *polling* (refresh periódico del feed); WebSocket/SSE para nuevas vistas/reacciones en vivo es explícitamente **fase futura**, no v1 (no hay librería de tiempo real en `requirements.txt`).
- **Prioridad / fase:** Alta / BACK-9.

30. Especificación de endpoints — Chat y mensajería

Estado actual: no existe ningún modelo ni endpoint. Dominio nuevo completo. Grupos quedan fuera de v1 por decisión ya tomada en el ciclo de producto.

Ficha: Conversaciones 1:1 y mensajes

- **Endpoints:** *Contrato recomendado, todavía no implementado.*
- `POST /api/v1/conversations` · Bearer · body `{participant_id: uuid}` · valida que no exista ya una conversación 1:1 entre ambos (idempotente: si existe, devuelve la existente) · 201/200, 403 (bloqueo activo), 422
- `GET /api/v1/conversations` · Bearer · paginado por cursor · devuelve resumen con último mensaje y no leídos
- `GET /api/v1/conversations/{id}/messages` · Bearer, solo participantes · paginado por cursor cronológico inverso
- `POST /api/v1/conversations/{id}/messages` · Bearer, solo participantes · body `{content, shared_resource: {type: "post"|"village"|"activity", id} (opcional), client_message_id (uuid, para idempotencia)}` · 201, 403, 413 (adjunto), 422
- `PATCH /api/v1/conversations/{id}/read` · Bearer · marca leídos hasta un `message_id`
- **Modelo/tabla:** `conversations (id, created_at)`, `conversation_participants (conversation_id, user_id, UNIQUE(conversation_id, user_id))`, `messages (id, conversation_id FK CASCADE, sender_id FK users, content, shared_resource_type, shared_resource_id, client_message_id UNIQUE, created_at, deleted_at)`, `message_receipts (message_id, user_id, read_at)`.
- **Idempotencia:** `client_message_id` único generado por el cliente antes de enviar — un reintento de red con el mismo id no duplica el mensaje (constraint `UNIQUE`).
- **Orden:** cursor estable por `(created_at, id)` descendente.
- **Privacidad/moderación:** solo participantes acceden (403 si no); bloqueo (§39) impide crear conversación nueva pero no borra el historial existente.
- **Tiempo real:** v1 REST + polling corto en la conversación activa; WebSocket/SSE es **fase futura explícita** (BACK-11) — no hay ninguna dependencia de tiempo real instalada hoy.
- **Retención/cifrado:** cifrado en tránsito ya cubierto por HTTPS; cifrado en reposo y política de retención quedan como decisión pendiente (§41), no bloquean v1.
- **Rate limiting:** por usuario en `POST /messages` (no existe rate limiting de ningún tipo hoy en el backend — requiere middleware nuevo, ver §36).
- **Prioridad / fase:** Alta (para desbloquear FRONT-7 operativo) / BACK-8.

31. Especificación de endpoints — notificaciones, comentarios y reacciones

Ficha: Notificaciones

- **Estado actual:** no existe ningún modelo ni endpoint.
- **Endpoints:** *Contrato recomendado, todavía no implementado.*
- `GET /api/v1/notifications` · Bearer · paginado por cursor · filtro `type`, `read`
- `GET /api/v1/notifications/unread-count` · Bearer · `{unread_count}`
- `PATCH /api/v1/notifications/{id}/read`
- `PATCH /api/v1/notifications/read-all`
- **Modelo/tabla:** `notifications` (`id`, `recipient_id` FK `users` CASCADE, `actor_id` FK `users` nullable, `type` enum, `resource_type`, `resource_id`, `read_at` nullable, `dedupe_key`, `created_at`). Índice (`recipient_id`, `read_at`, `created_at`).
- **Tipos:** `new_follower`, `post_like`, `post_comment`, `moment_reply`, `new_message`, `activity_reminder`, `village_news`.
- **Deduplicación:** `dedupe_key` único por (`recipient_id`, `type`, `resource_id`) en ventanas cortas, para evitar spam de notificaciones repetidas.
- **Generación:** cada dominio (likes, comentarios, follows, mensajes) emite un evento interno que un servicio común `NotificationService.notify(...)` convierte en fila — no cada endpoint escribe directamente en `notifications`.
- **Prioridad / fase:** Alta / BACK-3.

Ficha: Comentarios de publicaciones

- **Estado actual:** solo existe la columna muerta `comments_count` en `community_post.py`; sin tabla ni endpoint.
- **Endpoints:** `GET/POST /api/v1/posts/{post_id}/comments`, `PATCH/DELETE /api/v1/comments/{comment_id}` · *Contrato recomendado, todavía no implementado.*
- **Modelo/tabla:** `post_comments` (`id`, `post_id` FK CASCADE, `author_id` FK `users`, `content`, `created_at`, `updated_at`, `deleted_at`), índice (`post_id`, `created_at`).
- **Contador consistente:** `comments_count` en `community_posts` se actualiza dentro de la **misma transacción** que el insert/soft-delete del comentario (o se deriva por `COUNT()` en la respuesta si se prioriza simplicidad sobre rendimiento) — nunca incrementado desde el cliente.
- **Validaciones:** `content` 1-2000 caracteres; rate limit por usuario (nuevo middleware).
- **Prioridad / fase:** Alta / BACK-3.

32. Especificación de endpoints — publicaciones, tipos de contenido y pueblo enriquecido

Ficha: Publicaciones tipadas

- **Estado actual:** `community_post.py` tiene un único tipo implícito (`title`, `content`, `image_url`).
- **Ampliación recomendada:** añadir `type` enum (`"general"`, `"aviso"`) en v1 — **no** los 10 tipos originalmente listados por el negocio; el resto (foto/álbum/recuerdo/recomendación/pregunta/plan/experiencia) se cubren con `type="general"` + `content` libre hasta que haya evidencia de uso que justifique tipos adicionales. *Contrato recomendado, todavía no implementado* para la columna `type`.
- **Migración:** añadir columna `type` con default `"general"` a `community_posts`, sin romper filas existentes.
- **Validación adicional para `type="aviso"`:** podría requerir rol (vecino verificado, asociación) — decisión de producto pendiente (§41), no bloquea el resto.

Ficha: Pueblo enriquecido — campos simples vs. contenido editorial

- **Estado actual:** `village.py` ya tiene `description` (texto) y `highlights` (JSONB list[str]) — reutilizables sin cambios para los bloques 2 y 3 de §8.5.

- **Campos nuevos simples recomendados** (añadir a `villages`, una sola migración): `best_season` (string, nullable), `official_links` (JSONB, nullable), `accessibility_notes` (text, nullable), `transport_notes` (text, nullable).
- **Contenido editorial vs. generado por usuarios:** historia/cultura/gastronomía/patrimonio se resuelven con los campos simples anteriores (editorial, mantenido por admin vía el CRUD de pueblos ya existente) — **no requieren tablas nuevas**. "Voces de la comunidad" y "Fotos recientes" (bloques 7-8 de §8.5) son contenido generado por usuarios y se resuelven reutilizando `community_posts` filtrados por `village_id`, sin tabla nueva.
- **Endpoint:** ampliar el `VillageResponse` ya existente con los campos nuevos; no se recomienda un endpoint compuesto separado "detalle completo" — el patrón actual (un GET de pueblo + GETs paralelos de posts/activities ya usado en `app/villages/{id}/page.tsx`) es correcto y debe mantenerse, añadiendo un GET adicional solo para "pueblos relacionados" si se decide resolverlo en servidor en vez de en cliente (ver ficha siguiente).
- **Prioridad / fase:** Media / BACK-5.

Ficha: Pueblos relacionados (opcional, servidor vs. cliente)

- **Estado actual:** no existe. Puede resolverse en frontend filtrando el catálogo ya cargado (sin backend nuevo, ver §8.5 bloque 10).
- **Alternativa backend (solo si el catálogo crece más allá de lo que el frontend puede cargar):** `GET /api/v1/villages/{id}/related` · público · devuelve pueblos de la misma región/provincia excluyendo el actual, paginado. *Contrato recomendado, todavía no implementado — de prioridad baja, solo si §40 confirma que el frontend ya no puede resolverlo con datos locales.*

33. Especificación de endpoints — actividades avanzadas, búsqueda, guardados y pasaporte

Ficha: Actividades — estado, aforo y participantes

- **Estado actual:** `activities.py` / `activity.py` ya tienen `capacity`, `status` (default "published"), join/leave con `activity_participants`. **No existe** comprobación de aforo antes de duplicado (ya señalado en la auditoría técnica: el aforo se comprueba antes que la duplicidad), ni endpoint de lista de participantes, ni lista de espera.
- **Ampliación recomendada:** `GET /api/v1/activities/{id}/participants` (organizador/admin; público solo si se decide) — *contrato recomendado, todavía no implementado*; corregir el orden de comprobación aforo→duplicidad en el servicio existente (cambio de lógica, no de contrato); añadir `status` enum real con valores `published|cancelled|full` en vez de solo "published" por defecto.
- **Concurrencia:** el join debe ejecutarse dentro de una transacción con bloqueo (`SELECT ... FOR UPDATE` sobre la fila de la actividad o un `COUNT` transaccional) para evitar sobreaforo bajo carga concurrente — hoy no hay evidencia de ese control en el código auditado.
- **Prioridad / fase:** Alta / BACK-6.

Ficha: Búsqueda global

- **Estado actual:** cada catálogo tiene su propio filtro `search` de texto (`villages/activities/posts`); no existe endpoint unificado.
- **Endpoint:** `GET /api/v1/search?q=&types=villages,activities,posts,users&limit_per_type=&offset=` · público, Bearer opcional para personalización futura · *Contrato recomendado, todavía no implementado.*
- **Motor recomendado:** PostgreSQL full-text search (`tsvector` / `tsquery`) o trigramas (`pg_trgm`) para tolerancia a errores tipográficos — explícitamente **no Elasticsearch** en esta etapa, tal como se pidió, dado el volumen de datos actual (unidades de pueblos/actividades, no millones de filas).
- **Migración:** añadir columna `search_vector` `tsvector` (generada) + índice GIN en `villages`, `activities`, `community_posts`; extensión `pg_trgm` si se prioriza tolerancia a typos sobre relevancia semántica.
- **Response:** `{query, results: {villages[], activities[], posts[], users[]}, counts}`.
- **Prioridad / fase:** Media / BACK-7.

Ficha: Guardados — listado de colección

- **Estado actual:** POST/DELETE /posts/{id}/save y /activities/{id}/save ya existen y funcionan (tablas saved_posts , saved_activities). **Falta el GET de colección.**
- **Endpoints:** GET /api/v1/users/me/saved-posts , GET /api/v1/users/me/saved-activities · Bearer · paginado · *Contrato recomendado, todavía no implementado* (mutadores ya existen, solo falta el listado).
- **Prioridad / fase:** Alta / BACK-2.

Ficha: "Quiero visitar" y Pasaporte real

- **Estado actual:** no existe ninguna relación equivalente. **No debe reutilizarse SavedActivity / VillageFollower** para este significado — son relaciones con semántica distinta (seguir ≠ guardar ≠ querer visitar ≠ visitado).
- **Endpoints:** POST/DELETE /api/v1/villages/{id}/want-to-visit · Bearer · *Contrato recomendado, todavía no implementado*; GET /api/v1/users/me/passport devuelve {followed_villages, want_to_visit_villages, visited_villages} agregando las relaciones correspondientes.
- **Modelo/tabla:** village_want_to_visit (village_id, user_id, created_at, UNIQUE(village_id, user_id)) — misma forma que village_followers pero tabla independiente, para no mezclar significados.
- **"Visitado" (v2, fuera de v1):** requiere verificación — dos vías posibles a evaluar en el futuro: autodeclaración moderable (village_visits (village_id, user_id, declared_at, verified boolean default false)) o check-in por geolocalización (requiere permisos de ubicación en frontend y validación de proximidad en backend) — **explícitamente pospuesto**, no se especifica en detalle aquí por decisión ya tomada de evitar premiar solo cantidad de visitas sin verificación.
- **Prioridad / fase:** Media / BACK-2.

34. Modelos, tablas, relaciones e índices necesarios (resumen consolidado)

Tabla nueva	Propósito	Índices/restricciones clave
user_privacy_settings (o columnas en users)	Visibilidad por campo del perfil	—
user_interests	Intereses del usuario	(user_id, interest_tag_id)
user_followers	Seguimiento entre personas	UNIQUE(follower_id, followed_id)
user_onboarding_state (o columnas en users)	Progreso de onboarding	—
moments, moment_views, moment_reactions, moment_replies, moment_highlights	Dominio Momentos	índice en expires_at; UNIQUE por vista/reacción de usuario
conversations, conversation_participants, messages, message_receipts	Chat 1:1	UNIQUE(client_message_id); UNIQUE(conversation_id, user_id)
notifications	Notificaciones	índice (recipient_id, read_at, created_at)
post_comments	Comentarios	índice (post_id, created_at)
village_want_to_visit	Pasaporte — quiero visitar	UNIQUE(village_id, user_id)
media_assets	Multimedia (§37)	owner_id, status
Columnas nuevas en users	home_village_id, residence_village_id, show_home_village, onboarding_completed	FK ON DELETE SET NULL
Columnas nuevas en villages	best_season, official_links, accessibility_notes, transport_notes, search_vector	índice GIN en search_vector

Tabla nueva	Propósito	Índices/restricciones clave
Columna nueva en <code>community_posts</code>	<code>type</code> enum	default <code>"general"</code>

35. Migraciones futuras requeridas

En orden recomendado (cada una independiente y reversible):

1. `users`: `home_village_id`, `residence_village_id`, `show_home_village`, `interests / user_interests`, `onboarding_completed` — resuelve de paso el bug de `favorite_village_id`.
2. `user_followers`.
3. `user_privacy_settings` (o columnas booleanas en `users`).
4. `village_want_to_visit`.
5. `post_comments` + columna `comments_count` recalculada.
6. `notifications`.
7. `community_posts.type`.
8. `villages`: campos de información práctica + `search_vector` (GIN).
9. `media_assets`.
10. `moments` + tablas asociadas.
11. `conversations`, `conversation_participants`, `messages`, `message_receipts`.

36. Autenticación, autorización y privacidad

Reutilizar el esquema Bearer manual existente (`app/api/v1/endpoints/auth.py`) para todos los endpoints nuevos — no se recomienda migrar a `OAuth2PasswordBearer` solo por esta ampliación (cambiaría un contrato ya probado sin beneficio claro). Sí se recomienda, de forma transversal a todo lo nuevo: declarar `HTTPBearer` en OpenAPI (deuda ya señalada), aplicar `response_model` a cada endpoint nuevo desde el primer commit (evitar repetir la deuda de `app/schemas/common.py`), y verificar ownership en servidor para cada mutación (nunca confiar en `role / id` de cliente). Autorización por rol: reutilizar el patrón existente (`role == "admin"` inline) para lo nuevo que lo requiera (Momento tipo "aviso", moderación); no se justifica un sistema de permisos genérico para el volumen actual de reglas.

37. Multimedia y almacenamiento

No existe hoy ningún mecanismo de subida de archivos (`python-multipart` está en `requirements.txt` pero no se usa en ningún endpoint auditado) ni almacenamiento externo (`boto3` ausente). Contrato recomendado, todavía no implementado:

- `POST /api/v1/media` multipart · Bearer · body `file`, `purpose: "avatar"|"banner"|"post"|"moment"|"activity"` · response `{id, url, width, height, mime_type, size_bytes}` · 201, 401, 413 (tamaño), 415 (MIME), 422.
- `DELETE /api/v1/media/{id}` · Bearer, solo propietario.
- **Almacenamiento:** objeto compatible S3 (o equivalente) — **requiere una dependencia nueva** (`boto3` o similar) y credenciales de un proveedor, fuera del alcance de "sin instalar nada"; se documenta aquí como requisito futuro explícito, no se instala nada en esta tarea.
- **Validación:** MIME real (no solo extensión), límite de tamaño (ej. 8MB imagen), stripping de metadatos EXIF, generación de miniatura.
- **Modelo:** `media_assets` (`id`, `owner_id` FK users CASCADE, `purpose`, `url`, `width`, `height`, `mime_type`, `size_bytes`, `status`, `created_at`, `deleted_at`).

38. Tiempo real, eventos, notificaciones y jobs

No existe hoy ningún mecanismo de jobs programados ni tiempo real en el backend. Dos necesidades nuevas identificadas:

- **Job de expiración de Momentos** (§29): requiere un scheduler (ej. APScheduler o un cron externo que llame a un endpoint interno protegido) — dependencia nueva, no instalada en esta tarea.

- **Notificaciones en tiempo real / chat en vivo:** v1 se resuelve con *polling* desde el cliente (sin dependencia nueva); WebSocket/SSE queda como **fase futura explícita** (BACK-11), condicionado a evidencia de que el polling no sea suficiente.
- **Eventos internos:** cada dominio (likes, comentarios, follows, mensajes, reacciones) debe emitir un evento interno consumido por `NotificationService` — puede implementarse como llamada de función directa dentro de la misma transacción (sin cola de mensajes) mientras el volumen sea bajo; una cola (Redis/Celery) es explícitamente **fase futura**, no v1.

39. Moderación, seguridad y RGPD

- **Bloqueo entre usuarios:** `user_blocks (blocker_id, blocked_id, created_at, UNIQUE(blocker_id, blocked_id))` — *contrato recomendado, todavía no implementado*. Debe consultarse antes de: crear conversación, follow, comentar, reaccionar.
- **Reportes:** `reports (id, reporter_id, target_type ("post","comment","user","message","activity","moment"), target_id, reason, status ("pending","reviewed","dismissed"), created_at, reviewed_by, reviewed_at)` — endpoint `POST /api/v1/reports` genérico por tipo de recurso.
- **Panel de moderación:** `GET /api/v1/admin/reports`, `PATCH /api/v1/admin/reports/{id}` — solo admin, genera entrada de auditoría.
- **Auditoría:** `audit_logs (id, actor_id, action, target_type, target_id, metadata JSONB, created_at)` para toda acción administrativa (suspender usuario, resolver reporte, borrar contenido ajeno).
- **RGPD:** endpoints futuros `GET /api/v1/users/me/export` (exportación de datos) y `DELETE /api/v1/users/me` (baja con anonimización, no borrado físico inmediato de contenido de terceros que lo referencie) — **fase futura**, no bloquea v1 pero debe documentarse antes de tener usuarios reales en producción.
- **Rate limiting:** no existe ningún mecanismo hoy; requiere middleware nuevo (ej. `slowapi`) — dependencia nueva, no instalada en esta tarea. Aplicar como mínimo a login/registro, comentarios, mensajes, reportes.

40. Paginación, consistencia, concurrencia e idempotencia

Estándar recomendado para todo endpoint nuevo (y a extender a los existentes en una fase de estabilización ya señalada en la auditoría técnica previa):

```
{ "items": [...], "total": 0, "limit": 20, "offset": 0, "has_more": false }
```

- Orden estable: siempre un segundo criterio de desempate por `id` además de `created_at`.
- Timestamps UTC-aware (`DateTime(timezone=True)`) en toda tabla nueva — no repetir el problema ya señalado en las tablas existentes.
- Idempotencia explícita donde el cliente puede reintentar: envío de mensajes (`client_message_id`), reacciones/vistas de Momentos (`UNIQUE` por usuario), follow/unfollow (upsert lógico).
- Concurrencia: join de actividades y contadores (likes, comentarios) requieren transacción con bloqueo o recomputo atómico — no incrementar contadores con lecturas no transaccionales.
- Caché: no se recomienda introducir caché (Redis) en v1 — el volumen actual no lo justifica; queda como optimización futura condicionada a métricas reales.

41. Estrategia de pruebas backend

Hoy `tests/test_health.py` está vacío y `pytest` no está en `requirements.txt` — es una brecha transversal previa a cualquier funcionalidad nueva. Para cada dominio nuevo de este documento se recomienda como mínimo: prueba de autorización (usuario no autenticado, usuario sin permiso, propietario, admin), prueba de validación (payload inválido → 422, no 500), prueba de idempotencia donde aplique, prueba de paginación (páginas consecutivas sin duplicados/huecos), y prueba de contrato frontend-backend (el `response_model` declarado coincide con lo que el adapter frontend espera). Ninguna funcionalidad de este documento debe darse por terminada sin esta batería, replicando el criterio ya usado en la auditoría técnica previa de este mismo proyecto.

42. Roadmap backend, dependencias, orden de implementación y conclusión final

Roadmap backend (BACK-0 a BACK-11)

Fase	Objetivo	Reutiliza	Nuevo	Riesgo principal	Prioridad
BACK-0	Seguridad, contratos y fundamentos (secretos, seeds, <code>response_model</code> , <code>Depends(get_db)</code> consistente, OpenAPI Bearer)	Todo el backend actual	—	Bloquea producción si no se resuelve primero	Muy alta
BACK-1	Perfil público, privacidad por campo, seguimiento entre usuarios, onboarding	<code>GET /users/{id_or_username}</code> , <code>PUT /users/me</code>	<code>user_followers</code> , <code>user_privacy_settings</code> , <code>user_interests</code> , onboarding	Filtración de campos privados si la migración de privacidad llega después que el endpoint público ampliado	Muy alta
BACK-2	Colecciones guardadas (listado), "Quiero visitar" y Pasaporte v1	<code>saved_posts</code> , <code>saved_activities</code> , <code>village_followers</code>	<code>village_want_to_visit</code>	Confundir semánticas si no se separa de guardado/seguido	Alta
BACK-3	Comentarios y notificaciones	<code>community_posts.comments_count</code> (a corregir)	<code>post_comments</code> , <code>notifications</code>	Contador desincronizado si no se actualiza transaccionalmente	Alta
BACK-4	Multimedia y almacenamiento	—	<code>media_assets</code> + proveedor externo (dependencia nueva a autorizar)	Requiere decisión de proveedor y credenciales	Alta
BACK-5	Pueblo enriquecido (campos simples + reutilización de posts para voces/fotos)	<code>villages.description/highlights</code> , <code>community_posts</code>	Columnas nuevas en <code>villages</code>	Ninguno relevante	Media
BACK-6	Actividades avanzadas (participantes, concurrencia de aforo, estados)	<code>activities</code> , <code>activity_participants</code>	Endpoint de participantes, corrección de orden aforo/duplicidad	Sobreaforo si no se corrige la transacción	Media-alta
BACK-7	Búsqueda (PostgreSQL full-text/trigram)	Filtros <code>search</code> existentes	<code>search_vector</code> + índices GIN	Ranking pobre sin ajuste fino	Media
BACK-8	Chat 1:1	—	Dominio completo nuevo	Moderación/bloqueo debe llegar antes o junto	Media
BACK-9	Momentos	<code>media_assets</code> (BACK-4)	Dominio completo nuevo + job de expiración	Requiere scheduler nuevo	Media
BACK-10	Moderación y administración (bloqueo, reportes, panel admin, auditoría)	<code>role == "admin"</code> existente	<code>user_blocks</code> , <code>reports</code> , <code>audit_logs</code>	Debe preceder a la apertura pública de Chat/comentarios a gran escala	Alta (transversal, no al final)
BACK-11	Escalabilidad y servicios futuros (tiempo real WebSocket/SSE, caché, colas, RGPD export/delete)	—	Todo nuevo, condicionado a evidencia de necesidad	Sobre-ingeniería si se adelanta sin datos de uso	Baja

Nota de secuencia: BACK-10 (moderación) se lista en la posición 10 por convención de nomenclatura, pero su contenido de bloqueo/reportes básico debe estar disponible **antes o junto con** BACK-8 (Chat) y BACK-3 (comentarios), no después — se marca explícitamente como prioridad alta y no estrictamente última en la práctica.

Matriz de dependencias FRONT-X → BACK-X

Frontend	Backend necesario
FRONT-1 (Perfil público)	BACK-1
FRONT-2 (Página de pueblo enriquecida)	BACK-5
FRONT-3 (badges/filtro actividades)	Ninguno nuevo (ya soportado)
FRONT-5 (Configuración con privacidad)	BACK-1
FRONT-6 (Pasaporte operativo)	BACK-2
FRONT-7 (Chat operativo)	BACK-8, BACK-10
FRONT-8 (Momentos operativos)	BACK-9, BACK-4, BACK-10
FRONT-9 (Moderación visual operativa)	BACK-10
Comentarios (dentro de FRONT-3 ampliado)	BACK-3
Notificaciones con badge real	BACK-3
Búsqueda unificada real	BACK-7

Orden recomendado de implementación full-stack

1. BACK-0 (fundamentos/seguridad) — sin esto, nada más debe desplegarse a producción.
2. FRONT-0 en paralelo (no depende de backend nuevo).
3. BACK-1 + FRONT-1 (perfil público y privacidad) juntos — el frontend no debe activar el perfil público antes de que la privacidad por campo exista.
4. BACK-10 (bloqueo/reportes básico) antes de abrir cualquier superficie social nueva (comentarios, chat, Momentos) a más usuarios.
5. BACK-2/BACK-3 + FRONT-5/FRONT-6 (colecciones, pasaporte, comentarios, notificaciones).
6. BACK-4 (multimedia) — desbloquea fotografías reales de perfil y Momentos.
7. BACK-5 + FRONT-2 (pueblo enriquecido).
8. BACK-6 (actividades avanzadas) en paralelo, baja interferencia con lo anterior.
9. BACK-8/BACK-9 + FRONT-7/FRONT-8 (Chat y Momentos operativos) — los prototipos visuales (FRONT-7/8 en su versión de vista previa) pueden construirse en cualquier momento anterior sin esperar a esta fase.
10. BACK-7 (búsqueda) y BACK-11 (escalabilidad) al final, condicionados a volumen de datos real.

Riesgos, decisiones técnicas y preguntas pendientes

- ¿Se resuelve `favorite_village_id` dentro de la migración de `home_village_id` / `residence_village_id` (recomendado) o se mantiene como campo separado?
- ¿Proveedor de almacenamiento multimedia (S3, R2, Cloudinary u otro) — decisión de infraestructura fuera del alcance de este documento?
- ¿Momentos tipo "Aviso" requiere verificación de rol especial (asociación/comercio) desde v1, o se abre a cualquier usuario y se modera a posteriori?
- ¿Se prioriza WebSocket para Chat antes de tener evidencia de que el polling es insuficiente, o se espera a datos de uso (recomendado esperar)?
- ¿Multilingüe (FRONT-11) implica también contenido editorial de pueblos en varios idiomas, lo que afectaría al modelo de `villages` de BACK-5?

Conclusión general

El backend actual de `Api-ConextaPueblos` es una base correcta y reutilizable — autenticación, catálogos y CRUD social ya funcionan — pero carece por completo de la capa que hace de ConectaPueblos una red social entre personas: no hay seguimiento entre usuarios, comentarios, notificaciones, mensajería, multimedia real ni moderación. Ninguna de estas ausencias es un defecto del código existente; son dominios que sencillamente no se han construido todavía. La secuencia recomendada no es "backend primero, frontend después" ni al revés: es **fundamentos de seguridad** → **identidad entre personas (perfil público + privacidad)** → **confianza básica (bloqueo/reportes)** → **capas sociales (comentarios, notificaciones,**

colecciones) → **multimedia** → **enriquecimiento de contenido** → **experiencias diferenciadoras (Chat, Momentos)**, con el frontend avanzando en paralelo mediante prototipos visuales claramente etiquetados que nunca se activan como funcionales hasta que su backend correspondiente exista, se pruebe y se despliegue. Ese orden protege el activo más valioso que ya tiene el producto: la confianza de no mostrar nunca algo que no sea real.